

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1
«Реализация REST API на основе boilerplate»

Выполнил:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Взять любой бойлерплейт (в работе использован `express-typeorm-boilerplate` (<https://github.com/kantegory/express-typeorm-boilerplate>) из задания) и с учётом результатов ДЗ1 (проектирование базы данных) и ДЗ2 (проектирование API) реализовать:

- модели;
- представления;
- контроллеры.

В результате нужно продемонстрировать полностью работающее API согласно варианту.

Вариант: 1 – приложение для бронирования столиков в ресторанах.

Ход работы

1. Что это за программа

Назначение. REST API для бронирования столиков в ресторанах (вариант 1). Приложение хранит рестораны, их меню, столики и временные слоты, принимает брони и отзывы.

Кто пользуется:

Роль	Что доступно
гость (без токена)	справочники городов/кухонь, поиск ресторанов, страница ресторана, меню, фото, отзывы, свободные столики — всё только на чтение
USER	всё, что и гостю, плюс: бронирование, свой отзыв (один на ресторан), свой профиль (<code>/users/me</code>), своя история броней
RESTAURANT_ADMIN	управление ресторанами, на которые назначен (через таблицу <code>RestaurantAdmin</code>): сам ресторан, его фото, меню, столики, слоты, просмотр своих броней
ADMIN	всё: справочники, любые рестораны, назначение администраторов ресторанов, все брони системы

Что умеет, в шести группах:

1. вход и регистрация — `POST /auth/register`, `POST /auth/login`;
2. поиск ресторанов — `GET /restaurants` с фильтрами, справочники `GET /cities`, `GET /cuisines`;

3. страница ресторана — сам ресторан, меню (/menu), фото (/photos), отзывы (/reviews);
4. свободные столики — GET /restaurants/{id}/availability;
5. бронирование — POST /bookings, GET /bookings/{id}, PATCH /bookings/{id}/status, GET /users/me/bookings;
6. управление рестораном — создание/изменение ресторана, его столиков, слотов, меню, фото, назначение администраторов.

Стек и основа. Node 20, Express, TypeORM, PostgreSQL, JWT

(jsonwebtoken), поверх бойлерплейта преподавателя

express-typeorm-boilerplate

(<https://github.com/kantegory/express-typeorm-boilerplate>) (даёт связку

Express + TypeORM + routing-controllers и одну модель User — остальное дописано).

В цифрах: 4695 строк кода в src/, 43 операции (полностью совпадают со спецификацией openapi.yaml, см. раздел 9), 13 контроллеров, 11 таблиц.

2. Из чего состоит проект

Схема 1 — структура папок

src/	
— app.ts, swagger.ts	точка входа и документация
— config/	настройки и подключение к БД
— common/	инфраструктура контроллеров
— models/	11 сущностей + enums/ + user.subscriber.ts
— dto/	тела запросов и формы ответов
— controllers/	13 файлов — по одному на ресурс
— services/	бизнес-логика (booking, availability, review, restaurant-access)
— middlewares/	auth, roles, restaurant-admin, error-handler
— errors/	свой ConflictError
— utils/	мелкие переиспользуемые функции
— seeds/	наполнение базы демо-данными

config/ — настройки, общие для всего приложения: подключение к БД (data-source.ts) и переменные окружения в одном месте (settings.ts), чтобы нигде больше не читать process.env напрямую.

common/ — инфраструктура самих контроллеров (обязка вокруг routing-controllers), а не бизнес-логика конкретного ресурса: базовый класс BaseController и декоратор @EntityController.

models/ — сущности TypeORM (*.entity.ts), которые превращаются в таблицы, плюс enum-ы (models/enums/) и подписчик, который хэширует пароль перед сохранением пользователя.

dto/ — «представления»: классы, описывающие форму тела запроса (с валидацией) и форму ответа. Именно здесь `entity` превращается в JSON, который видит клиент.

controllers/ — один файл на ресурс, отвечает только за HTTP-часть: разбор запроса, проверка прав, вызов сервиса или репозитория, формирование ответа.

services/ — бизнес-логика, которая не помещается в один метод контроллера: поиск свободных столиков, проверки при создании брони, пересчёт рейтинга ресторана.

middlewares/ — код вокруг контроллера: одни проверяют запрос до него (токен, роль, доступ к ресторану) и могут его прервать, другие перехватывают ошибку уже после.

errors/ — класс `ConflictError` (409), которого не хватает в готовом наборе `routing-controllers`.

utils/ — маленькие функции без своего состояния, переиспользуемые в разных слоях (пароли, пагинация, форматирование, коды ошибок Postgres).

seeds/ — отдельный скрипт заполнения БД для демонстрации, не часть работающего сервера.

Словарик

- **контроллер** — класс, который принимает HTTP-запрос и решает, что с ним делать дальше;
- **сервис** — функция или набор функций с бизнес-логикой, которая не привязана к HTTP;
- **модель (entity)** — класс, описывающий одну таблицу базы данных;
- **DTO** — класс, описывающий форму данных на входе (с проверкой) или на выходе (что видит клиент);
- **middleware** — код, который выполняется вокруг контроллера и может пропустить запрос дальше или прервать его.

3. Как программа запускается

`app.ts` при старте, по шагам:

1. читает настройки из `SETTINGS` (`config/settings.ts`);

2. создаёт Express-приложение, подключает `cors()` и `express.json()`;
3. регистрирует 13 контроллеров и `ErrorHandlerMiddleware` через `useExpressServer(app, options)`;
4. подключает страницу документации `/docs` через `useSwagger(app, options)`;
5. запускает `dataSource.initialize()` (соединение с базой) и `app.listen(...)` — оба вызова асинхронные и не ждут друг друга: сервер физически начинает слушать порт, даже если соединение с базой ещё не установилось. На практике это не мешает, потому что реальный первый запрос приходит уже после того, как оба шага успевают завершиться, но строго последовательным этот порядок не назвать.

Настройки. `.env` → `settings.ts`: каждое поле класса `Settings` читает `process.env.ЧТО_ТО`, а если переменной нет — берёт значение по умолчанию прямо в коде. Больше нигде в проекте `process.env` не читается напрямую.

База данных. Контейнер PostgreSQL из `docker-compose.yml`. Запуск одной строкой:

```
docker compose up -d
```

4. Слои и как файлы работают вместе

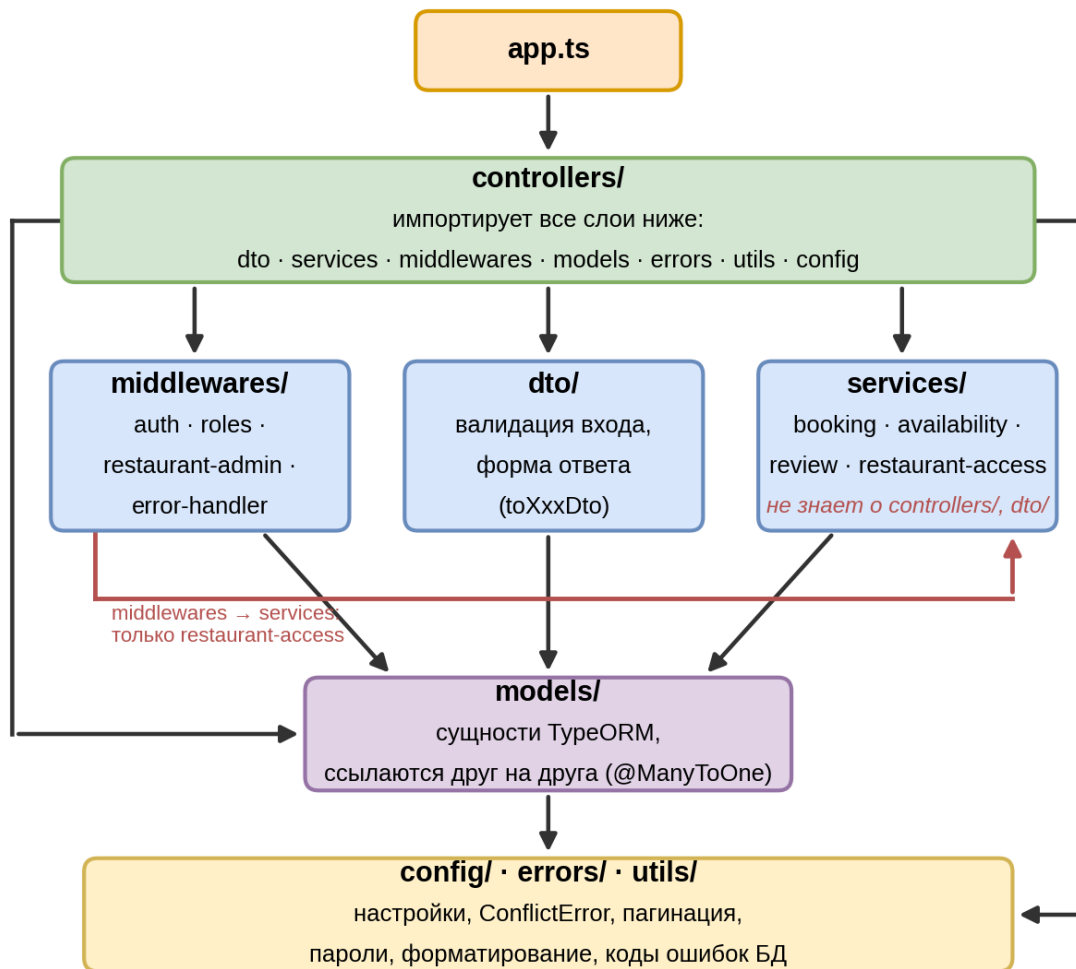
Слой	Что делает	Чего не делает
Контроллер	принимает запрос, проверяет право доступа через <code>middleware</code> , вызывает репозиторий или сервис, оборачивает результат в DTO	не содержит сложную бизнес-логику, не собирает сырой SQL сам
DTO	проверяет входящие данные (<code>class-validator</code>), описывает форму ответа, превращает Entity в JSON	не обращается к базе, не принимает решений о правах доступа
Сервис	держит бизнес-логику, которая не умещается в один метод контроллера (проверки, транзакции, непростые запросы)	не знает о HTTP (<code>Request/Response</code>), не формирует ответ клиенту
Модель (entity)	описывает таблицу и её ограничения декораторами <code>TypeORM</code>	не содержит логики, только объявляет связи с другими моделями

Слой	Что делает	Чего не делает
Middleware	решает, пускать ли запрос дальше (401/403), либо перехватывает ошибку уже после контроллера	не выполняет бизнес-логику самого ресурса

Правила зависимостей — сверху вниз. Я проверила это по реальным импортам (не предположение): ни один файл в `services/` не импортирует ничего из `controllers/` или `dto/` — сервисы берут только `config/`, `models/`, `errors/`, `utils/`. Модели импортируют только другие модели (через связи) и `utils/` (`numericTransformer`). Контроллеры — единственный слой, который импортирует все остальные (`dto`, `services`, `middlewares`, `models`, `errors`, `utils`). Обратной зависимости — «сервис знает о конкретном контроллере» — в проекте нет нигде.

Роль `@EntityController` и `BaseController`. `BaseController` (`common/base-controller.ts`) просто объявляет, что у наследника будет поле `repository`. Декоратор `@EntityController({ baseRoute, entity })` (`common/entity-controller.ts`) при применении к классу: (1) вызывает стандартный `JsonController(baseRoute)` из `routing-controllers`, регистрируя маршруты; (2) если передан `entity` — добавляет геттер `repository`, который при каждом обращении берёт `dataSource.getRepository(entity)`. Поэтому в контроллере можно сразу писать `this.repository.find(...)`, не создавая репозиторий вручную.

Схема 2 — взаимодействие файлов (кто кого импортирует)



Примечание: единственное исключение из направления «сверху вниз» — `utils/pagination.ts` берёт `mun PageMetaDto` из `dto/pagination.dto.ts`.

5. База данных в коде

11 таблиц и их связи перенесены из диаграммы ДЗ1: `City`, `Cuisine` — справочники; `Restaurant` — центральная сущность; `RestaurantAdmin`, `RestaurantPhoto`, `MenuItem`, `RestaurantTable`, `TimeSlot` — зависят от ресторана; `Review` (ссылается на ресторан и пользователя) и `Booking` (ссылается на пользователя, столик и слот). Плюс три `enum`-а: `Role`, `BookingStatus`, `MenuCategory` (`models/enums/`).

Как ограничения из ДЗ1 перенесены в TypeORM:

- `UNIQUE` — обычный `@Unique([...])` (например, `RestaurantAdmin: ['user_id', 'restaurant_id'], Review: ['user_id', 'restaurant_id'], RestaurantTable:`

- ```
['restaurant_id', 'number'], TimeSlot: ['restaurant_id', 'start_time']);
```
- CHECK — декоратор `@Check(...)` прямо с SQL-условием внутри (например, `"rating" >= 0 AND "rating" <= 5` на `Restaurant`, `"guests_count" > 0` на `Booking`);
  - частичные индексы — `@Index(..., { unique: true, where: '...' })`: одно главное фото на ресторан (`RestaurantPhoto`, условие `is_main = true`) и запрет двойной активной брони (`Booking`, условие `status IN ('PENDING', 'CONFIRMED')`) — оба ограничения работают, только пока выполняется условие в `where`, а не всегда;
  - ON DELETE RESTRICT — там, где данные — история и терять их нельзя: город/кухня ← ресторан, ресторан/пользователь ← отзыв, пользователь/столик/слот ← бронь;
  - ON DELETE CASCADE — там, где данные зависимые и бессмысленны без хозяина: ресторан → его фото/меню/столики/слоты/записи об администраторах.

**Особенности типов.** Колонки `numeric` (цена, рейтинг, координаты) драйвер `pg` возвращает строкой — за перевод в число отвечает `numericTransformer` (`utils/numeric-transformer.ts`), подключённый как `transformer` у каждой такой колонки. Колонки `time` возвращаются как `"HH:MM:SS"`, а нужен `"HH:MM"` — обрезкой занимается `formatTime` (`utils/time-format.ts`), вызывается в DTO-мапперах (`toRestaurantResponseDto`, `toTimeSlotDto`).

**Как создаются таблицы.** `synchronize: true` в `config/data-source.ts` — `TypeORM` сам сравнивает модели с текущей структурой базы и создаёт/меняет таблицы при каждом старте. Для учёбы это удобно: не нужно писать и поддерживать SQL-миграции вручную. В реальном проекте так не делают — на боевой базе с данными автоматическое изменение схемы слишком рискованно (можно случайно потерять данные при несовместимом изменении), поэтому там миграции пишут и проверяют отдельно.

## 6. Безопасность

**Пароль.** Хранится только как bcrypt-хэш. Хэширует его подписчик `UserSubscriber.beforeInsert (models/user.subscriber.ts)` прямо перед тем, как строка уйдёт в базу — контроллеры (`auth.controller.ts`) передают пароль как обычный текст и не занимаются хэшированием сами.

**Токен.** JWT с полезной нагрузкой `{ user: { id, role } }`, подписывается в `AuthController.issueTokenResponse (auth.controller.ts)`. Срок жизни задаётся переменной `JWT_ACCESS_TOKEN_LIFETIME` в `.env`.

**Два уровня проверки на каждом защищённом маршруте:**

1. `authMiddleware (middlewares/auth.middleware.ts)` — «кто вы»: проверяет подпись JWT, если токена нет или он невалиден — 401, дальше запрос не идёт;
2. проверка прав — «что вам можно», двух видов:  
`requireRole(...roles) (middlewares/roles.middleware.ts)` — по роли из токена, и `restaurantAdminMiddleware (middlewares/restaurant-admin.middleware.ts)` — «администратор именно этого ресторана», через сервис `isRestaurantAdmin (services/restaurant-access.service.ts)`. Обе — 403 при отказе.

**Роль и таблица `RestaurantAdmin`.** Связь пользователь↔ресторан хранится отдельной таблицей `restaurant_admin`, а не полем на пользователе — потому что администраторов у ресторана может быть несколько, и один человек может администрировать несколько ресторанов. При назначении (`POST /restaurants/{id}/admins, restaurant-admin.controller.ts`) пользователю с ролью `USER` роль автоматически меняется на `RESTAURANT_ADMIN` — это единственное место в коде, где роль пользователя меняется не при регистрации.

**Что сознательно сделано безопасно:**

- при входе одно и то же сообщение и на несуществующий email, и на неверный пароль (`auth.controller.ts:85`) — чтобы по ответу нельзя было узнать, зарегистрирован ли такой email;

- пароль и его хэш никогда не попадают в ответ API — DTO-мапперы (`toUserResponseDto` и т.п.) вручную перечисляют поля для ответа, а не отдают `entity` целиком.

## 7. Логика приложения

**Бронирование.** Порядок проверок в `createBooking` (`services/booking.service.ts`):

1. столик существует и активен → иначе 404;
2. слот существует и активен → иначе 404;
3. столик и слот принадлежат одному ресторану → иначе 400;
4. `guests_count` не больше вместимости столика → иначе 400;
5. дата не в прошлом → иначе 400;
6. на этот столик/слот/дату нет уже активной брони (SQL-запрос) → иначе 409.

Допустимые переходы статуса заданы картой `ALLOWED_TRANSITIONS` (`booking.service.ts`): `PENDING → CONFIRMED | CANCELLED`, `CONFIRMED → COMPLETED | CANCELLED`, `CANCELLED/COMPLETED` — конечные. Двойная защита от дублей: проверка в коде (пункт 6 выше) и частичный уникальный индекс в базе (`models/booking.entity.ts`) — на случай, если два запроса пройдут проверку одновременно, вторую вставку отклонит уже сама база (код ошибки 23505), и сервис поймает это как ещё один 409.

**Свободные столики.** Идея запроса в `getAvailableTables` (`services/availability.service.ts`): `LEFT JOIN` столиков с бронями на нужные дату/слот/активные статусы, и оставить только те строки, где `booking.id IS NULL` — то есть столик, для которого не нашлось ни одной подходящей брони.

**Поиск ресторанов.** В `RestaurantController.list` (`restaurant.controller.ts`) условия в `QueryBuilder` добавляются по одному `andWhere(...)`, только если соответствующий фильтр реально пришёл в query-параметрах (`q`, `city_id`, `district`, `cuisine_id`, `min_price`, `max_price`, `min_rating`). Сортировка берётся из карты `SORT_OPTIONS` по значению `sort`, пагинация — через `getSkipTake/buildPageMeta` (`utils/pagination.ts`).

**Отзывы.** Создание отзыва и пересчёт рейтинга ресторана происходят в одной транзакции (`dataSource.transaction(...)` в `review.controller.ts`, сам пересчёт — `recalculateRating` в `services/review.service.ts`), так что рейтинг никогда не может «отстать» от реального списка отзывов, даже если что-то пойдёт не так между двумя операциями.

## 8. Обработка ошибок

Единый формат `{statusCode, message, errors?}` формирует `ErrorHandlerMiddleware` (`middlewares/error-handler.middleware.ts`) — он подключён в `app.ts` как обработчик, срабатывающий после контроллера при любой брошенной ошибке.

**Откуда берутся коды:**

- **400** — от проверки DTO (`class-validator`, автоматически до вызова метода контроллера) и от сервисов при нарушении бизнес-правил (`BadRequestError` в `booking.service.ts`, например «дата в прошлом»);
- **401 / 403** — от `middleware` (`auth.middleware.ts`, `roles.middleware.ts`, `restaurant-admin.middleware.ts`);
- **404 / 409** — от сервисов и контроллеров: `NotFoundError` встроен в `routing-controllers`, а вот `ConflictError` (409) — свой класс (`errors/http-errors.ts`), потому что в библиотеке его не было.

**Ошибки базы.** `utils/db-errors.ts` умеет распознавать код 23503 (нарушение внешнего ключа) и 23505 (нарушение уникальности) в объекте ошибки Postgres. Там, где это нужно, операция оборачивается в `try/catch`: при удалении города, кухни, ресторана, столика и слота (23503), при создании и изменении слота (23505) и при создании брони (23505, в `booking.service.ts`). Пойманная ошибка превращается в понятный `ConflictError` (409) вместо голой ошибки SQL.

## 9. Документация и соответствие Д32

`/docs` показывает готовый файл `openapi.yaml` (`swagger.ts` читает его и отдаёт через `swagger-ui-express`), а не генерируется из кода. Так сделано

по двум причинам: автогенератор (`routing-controllers-openapi`) падал на части декораторов из-за особенностей `tsx`, и вдобавок так гарантируется, что документация 1-в-1 совпадает со спецификацией, по которой оценивают работу.

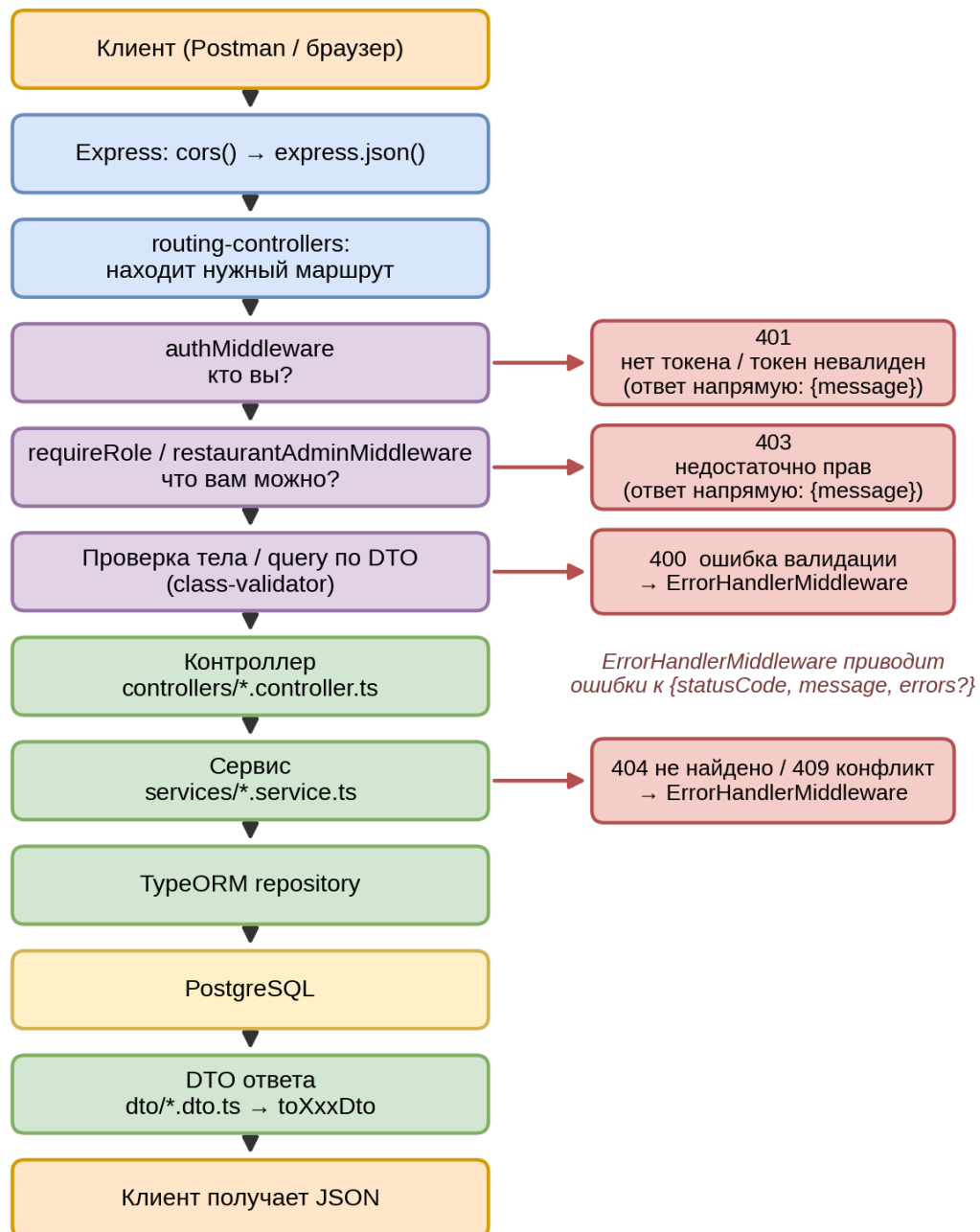
**Сверка.** 43 операции из 43 совпадают с `openapi.yaml` по адресам и HTTP-методам — проверено маршрут за маршрутом.

**Честно про отличия** (у каждого — как исправить одной строкой):

- 401 и 403 от `auth.middleware.ts/roles.middleware.ts/restaurant-admin.middleware.ts` приходят как `{message}`, без `statusCode` — потому что это обычные Express-middleware, а не действия `routing-controllers`, и они не проходят через `ErrorHandlerMiddleware`. **Исправить:** формировать в этих трёх файлах тот же объект `{statusCode, message}`, что и `ErrorHandlerMiddleware`.
- `minProperties: 1` из спецификации (шесть `*Update`-схем) не проверяется в коде — пустое тело `{}` в PATCH проходит валидацию. **Исправить:** добавить в эти DTO собственный валидатор «хотя бы одно поле заполнено».
- часть сообщений об ошибках — на английском (`'Restaurant not found'` и т.п.), хотя спецификация приводит примеры на русском. **Исправить:** перевести тексты в `throw new ...Error(...)` на русский.
- `GET /restaurants/{restaurantId}` не проверяет `is_active`, в отличие от списка `GET /restaurants`. **Исправить:** добавить то же условие `is_active = true` в `getOne` контроллера ресторана.

## 10. Путь запроса через программу

Схема 3 — сверху вниз



Порядок именно такой, потому что middleware из `@UseBefore(...)` выполняются до обработчика маршрута, а тело и query разбираются и проверяются при подготовке аргументов метода контроллера, то есть позже: сначала выясняется, кто вы и можно ли вам, и только потом проверяется, что вы прислали.

Любая ошибка на этой схеме, кроме 401/403 от middleware (см. раздел 9), доходит до `ErrorHandlerMiddleware` и приводится к формату `{statusCode, message, errors?}`.

За основу схемы взят маршрут создания брони (`POST /bookings`) — он проходит через все перечисленные шаги, включая проверку прав, сервисную логику и обращение к базе. Поиск ресторанов и вход идут по той же цепочке, только короче: без проверки прав (оба маршрута открыты без токена) и без сложной сервисной логики (данные читаются или пишутся прямо в контроллере).

## Вывод

В итоге получилось рабочее REST API из 43 операций, полностью покрывающее вариант «бронирование столиков», с чёткой слоистой структурой: контроллеры не знают деталей бизнес-логики, сервисы не знают о HTTP, модели описывают только структуру таблиц. Стало понятнее, как декораторы `routing-controllers` и `TypeORM` на самом деле связывают классы с маршрутами и таблицами через `reflect-metadata`, и почему в паре мест (см. раздел 9) пришлось выбирать между «как в спецификации» и «как получилось в коде», а не просто верить, что они совпадают. Дальше — по плану курса: в ДЗ3 API проверяется через Postman-коллекцию, а в ЛР2 предстоит разбить это же приложение на микросервисы, и самое сложное место там — `Booking`, у которой внешние ключи сразу на `restaurant_table` и `time_slot`, то есть бронь как сущность зависит от двух разных частей системы, которые в микросервисной архитектуре могут оказаться в разных сервисах.